# Semantic Embedding of Process Algebras in Event-B

GDR Scilog - *Webinaire Défi IPP* - 4 November 2026

Christian Attiogbé

Laboratoire des Sciences du Numérique de Nantes

Bienvenue ! - Welcome!



# Modeling and analysis of complex systems

- Formal models are efficient to describe and analyse systems of interest (whether natural systems or industrial systems).
- (Complex)Systems are heterogeneous by their nature.

**Various formalisms and tools to fit with needs ...**

Discrete/Continuous/Hybrid dynamic systems

☞ Among the challenges: composition and interoperability

# Agenda

1. Context (focus)
2. Overview of Process Algebras - CSP
3. Overview of the embedding policy
4. Overview of Event-B
5. Embedding method
6. Implementation

# Focus: interoperability of discrete event systems

*A discrete event system is a dynamic system with discrete states, the transitions of which are triggered by events.*

Various formalisms and tools are available for DES.

- ▷ Automata, Petri Nets
- ▷ Process Algebras
- ▷ Event-B
- ▷ ...

**Interoperability?!**

can be considered easy, but –

in practice not always the case –

☞ Process algebras (seamlessly) embedded in Event-B? $B[P]$

# Modeling and analyis of discrete event systems

**Explicit state-based**

Automata - Petri Nets
Process Algebras
+ Data, Time, Probabilities,
...

Analysis: **model checking**

**Logic/property-based**

FOL, Set Theory,
Type systems, HOL
Modal, Non-Linear logics,
...

Analysis: **theorem proving**

☞ this depends not only on the type of systems, but also on the abstraction levels, proficiency with tools

# Process Algebras

Process algebras are formalisms introduced by
A. W. Hoare (CSP, 1978),
R. Milner (CCS, 1980, $\pi$-calculus),
J. Bergstra (ACP, 1982)
to study interacting, concurrent and distributed systems.
ISO 8807 (Lotos 1989), Garavel (LNT 2017), $\cdots$

# Process Algebras

Process algebras are formalisms introduced by
A. W. Hoare (CSP, 1978),
R. Milner (CCS, 1980, $\pi$-calculus),
J. Bergstra (ACP, 1982)
to study interacting, concurrent and distributed systems.
ISO 8807 (Lotos 1989), Garavel (LNT 2017), $\cdots$

A **process** ($P$): abstraction to describe ***observable*** **behaviours**.

Sophisticated automata : (**State** space, **transitions**)

Behaviours of $P$ are described/observed through:
▷ elementary actions events: **alphabet** ($A_P$),
▷ a set of **operators** (to define and compose processes).

# (Focus on) CSP Operators

Consider $P$, $Q$ as processes, and $a \in A_Q$ , $b \in A_Q$.

- SKIP (normal termination); STOP (deadlock)
- **Prefix** the term $a \rightarrow P$ describes a process (say $Q$) which first engages in the event $a$ and then behaves as the process $P$.
  This is often expressed as $Q = a \rightarrow P$
  brackets are allowed; for instance $Q = (a \rightarrow (b \rightarrow P))$
- Two choice operators:
  the **external choice** ($P$ [] $Q$); it behaves like P or like Q depending on events (offered) from the **environment**.

  the **internal choice** ($P \sqcap Q$); the choice does not depend on the environment; the choice of $P$ or $Q$ is **non-deterministic**.

The choice operators are commutative and associative.

# CSP Operators

**Parallel composition, communication and synchronisation**

- Parallel processes can **synchronise** and communicate with their *shared events* (the intersection of their individual alphabets) $P \ |E| \ Q$ denotes parallel composition with synchronisation on events in the set $E$, with $E \subseteq A_P \cap A_Q$
- $P \ || \ Q$ denotes the parallel composition with **interleaving behaviour**; the alphabets are disjoint.

The parallel composition is commutative and associative.

# CSP Operators

**Hiding by abstraction** A behaviour can be made more abstract by hiding in it, the occurrence of certain events of its alphabet; from an observational point of view, these events do not occur. The term $P \backslash E$ (where $E \subseteq A_P$) denotes a process with the hiding of the events in $E$ from the observed behaviour of $P$.

**Recursion** The description and then the behaviour of a process can be recursive. For instance $P = a \rightarrow P$. But, the reference to the process itself can appear at any level of the behaviour, including through the other operators.

**Sequence ...**

# Summary - CSP Syntax

Given types: Process, Action (of the alphabet)

| Abstract syntax | | |
|---|---|---|
| Process | ::= | SKIP |
| | \| | STOP |
| | \| | Action → Process |
| | \| | Process ⌈] Process |
| | \| | Process [] Process |
| | \| | Process $\|E\|$ Process |
| | \| | Process \|\| Process |
| | \| | ... |

# Examples of processes behaviour

a process named *myP*, with $A_{myP} = \{a1, a2, b2, b3\}$:

$$myP = a1 \rightarrow (a2 \rightarrow (a2 \rightarrow skipP || a2 \rightarrow (b2 \rightarrow skipP)) \Box b3 \rightarrow skipP)$$

a process named *Clt*: (tries a connection and then interacts...)

$$Clt = lin \rightarrow C1$$
$$C1 = passw \rightarrow C2$$
$$C2 = (ko \rightarrow C1 \Box ok \rightarrow gC3)$$
$$gC3 = (com1 \rightarrow gC3 \Box com2 \rightarrow gC3 \Box com3 \rightarrow gC3) \Box lout \rightarrow Clt$$

# CSP Semantics - Structural operational semantics

**Prefix**: its semantic rule is

$$\frac{}{(\alpha \rightarrow Q) \xrightarrow{\alpha} Q} \ (prefixRule)$$

A reformulation of the same rule as the behaviour of $P$.

$$\frac{P = \alpha \rightarrow Q}{P \xrightarrow{\alpha} Q} \ (prefixRule)$$

# CSP Semantics - internal Choice

$$\frac{Q = (\beta \rightarrow rQ)}{(P \sqcap Q) \xrightarrow{\beta} rQ} \qquad (intChoiceRight)$$

the same rule for P, with the reformulation

$$\frac{P = (\alpha \rightarrow rP)}{(P \sqcap Q) \xrightarrow{\alpha} rP} \qquad (intChoiceLeft)$$

In fact $(P \sqcap Q) \xrightarrow{\tau} P \xrightarrow{\alpha} rP$

There are additional rules for the case where $P \xrightarrow{\tau} P'$

# CSP Semantics - other operators

There are similar semantic rules for the remaining CSP operators

| | |
|---|---|
| Process [] Process | extChoiceP semantic rules |
| Process $\|E\|$ Process | synParallelP semantic rules |
| Process \|\| Process | intParallelP semantic rules |

Can the CSP semantic rules be expressed in Event-B?

# Interoperability with CSP components

▷ Semantics of CSP encoded in ... (Event-B ?) 😎
(Encode the abstract syntax and the semantic rules 😟)

▷ Any CSP model can be expressed with the encoding formalism 😟
(and the same the other way)

▷ A semantic-preserving encoding of CSP with relations 😎
(convenient for semantic rules and can be embedded in Event-B)

# Overview of the proposed method

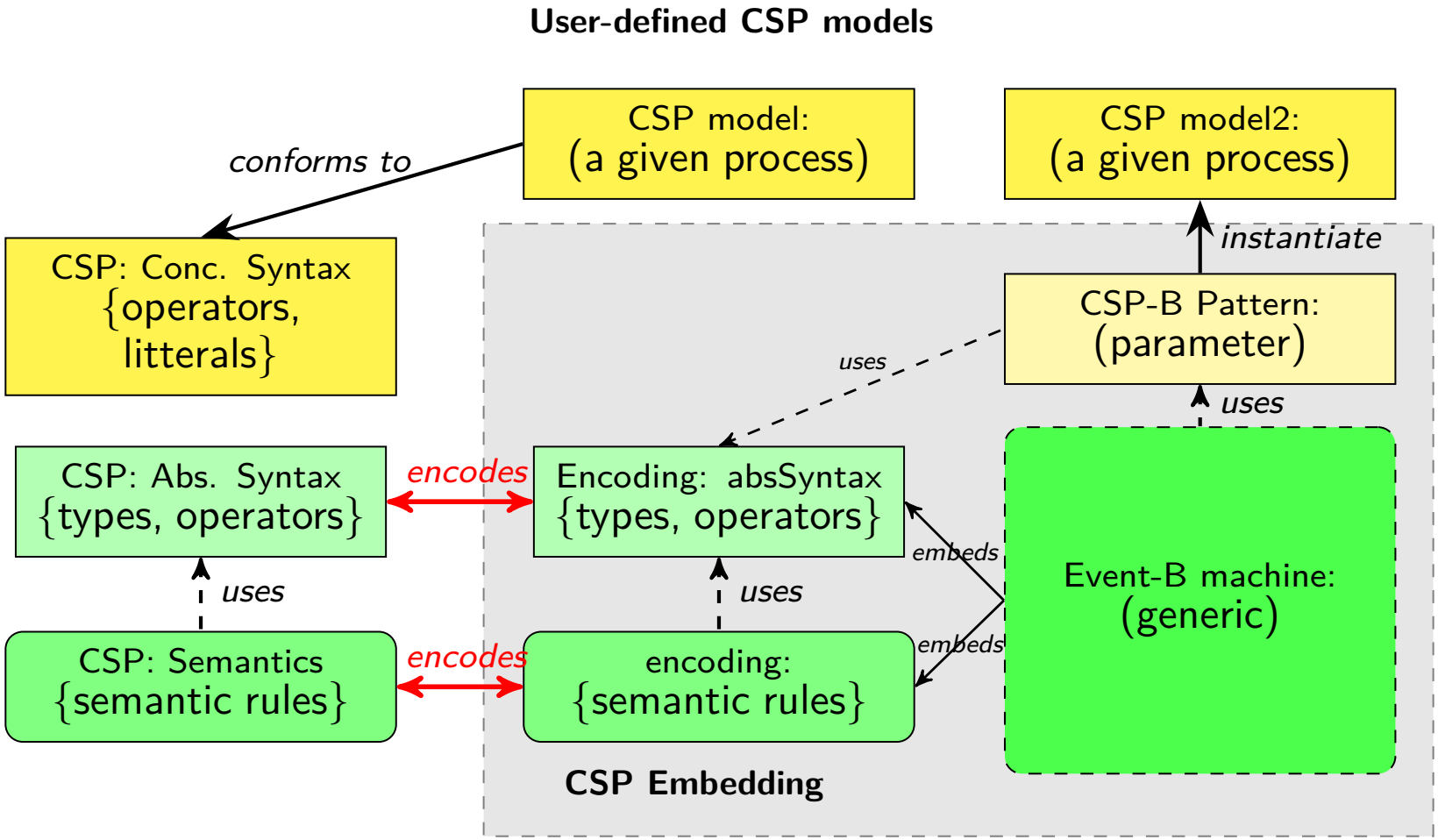



Figure: An overview of the method

# Abstract Syntax of CSP (revisited)

| Abstract syntax | | | Textual syntax |
|---|---|---|---|
| Process | ::= | SKIP | skipP |
| | \| | STOP | stopP |
| | \| | a → Process | prefP(a, Process) |
| | \| | Process ⌈] Process | intChoiceP(Process, Process) |
| | \| | Process [] Process | extChoiceP(Process, Process) |
| | \| | Process $\|E\|$ Process | synParallelP(Process, Process, E) |
| | \| | Process \|\| Process | intParallelP(Process, Process) |
| | \| | ... | ... |

☞ will be used for the embedding

# Event-B (by J-R. Abrial)

A formalism and a method for the correct-by-construction of discrete dynamic systems.

**Formalism:** Logic + set theory + Generalised substitution
**Reasoning:** Formal system (theorem prover)
**Construction:** Refinement theory
Components/systems are modelled with abstract/concrete machines.

# Event-B (by J-R. Abrial)

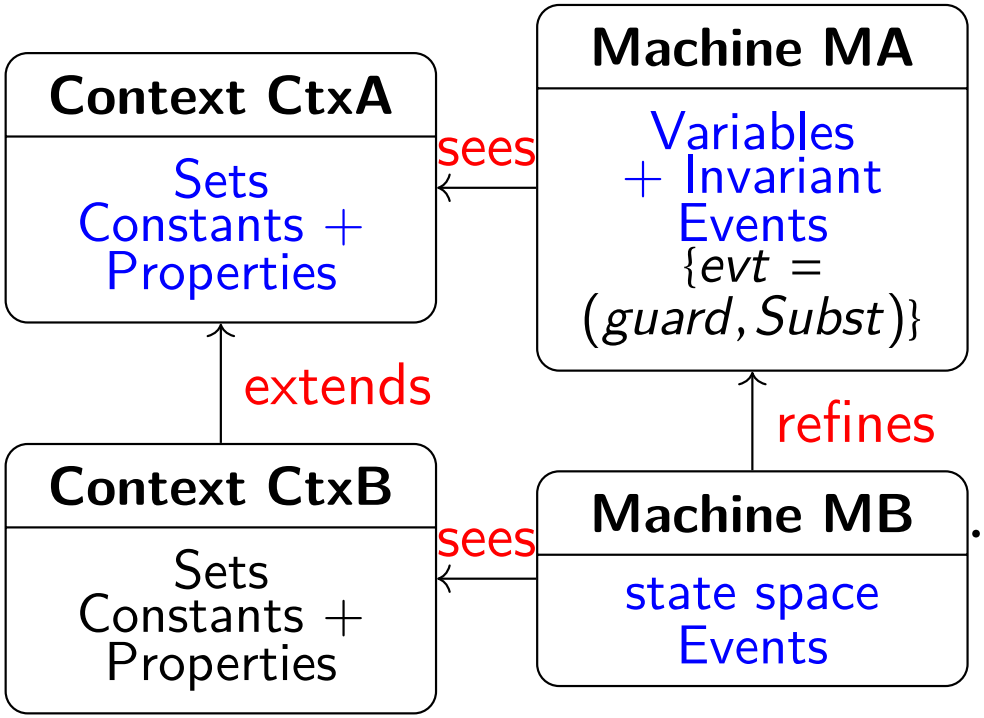
A formalism and a method for the correct-by-construction of discrete dynamic systems.

**Formalism:** Logic + set theory + Generalised substitution
**Reasoning:** Formal system (theorem prover)
**Construction:** Refinement theory
Components/systems are modelled with abstract/concrete machines.

An **abstract machine** simulates the behaviour of a system with a transition system.

A **state space** is described with invariant logical properties.
The **transitions** of the system relate the correct states of the space.
The transitions are modelled with **guarded events+substitutions**.

# Event-B modeling approach: a glimpse




```
MACHINE MA
SEES CtxA
VARIABLES x,y,z,f,r, ...
INVARIANTS
    r : E ↔ E
    f : (E × A) → E
    ...
EVENTS
    init = Subst
    evt1 = when guard
               then Subst end
    ...
END
```

# Relational Encoding of Process Algebras

**Prefix** The term $\alpha \rightarrow P$ of a process guarded by a prefix $\alpha$ (with $\alpha \in A_P$) is encoded with a function named here *prefP*.

$$prefP : Action \times Process \rightarrow Process$$

The elements of the function *prefP* are of the form $((\alpha, P),\ Q)$.
The behaviour $\alpha \rightarrow P$ is expressed with $prefP(\alpha, P)$

It follows that $\boxed{Q = \alpha \rightarrow P \text{ is in extension } prefP = \{((\alpha, P),\ Q)\}}$

# Internal and External Choice

The internal choice is encoded with

$$intChoiceP : Process \times Process \rightarrow Process$$

Similarly to the prefix function, we encode a process

$R = intChoiceP(P, Q)$ in extension with $intChoiceP = \{((P, Q),\ R)\}$

$P$ and $Q$ are encoded inductively.

The external choice is similar

$$extChoiceP : Process \times Process \rightarrow Process$$

# Synchronous and Interleaving Parallel

As for the **synchronous parallel composition**, we first consider that the two processes share the same alphabet.

$$synParallelP : Process \times Process \rightarrow Process$$

A process $R = synParallelP(P, Q)$ is e,coded with $synParallelP = \{((P, Q),\ R)\}$.

The **interleaving parallel composition** has a similar encoding.

$$intParallelP : Process \times Process \rightarrow Process$$

# Illustration of the encoding

The process

$$myP = \quad a1 \rightarrow (a2 \rightarrow (a2 \rightarrow skipP \parallel a2 \rightarrow (b2 \rightarrow skipP)) \\ \Box \\ b3 \rightarrow skipP)$$

is encoded as follows:

$$\begin{aligned} prefP &= \{((b3, skipP), P0),\ ((b2, skipP), P1),\ ((a2, P1), P2), \\ &\quad ((a2, skipP), P3),\ ((a2, P4), P5),\ ((a1, P6), \mathbf{myP})\} \\ intChoiceP &= \{((P5, P0), \mathbf{P6})\} \\ intParallelP &= \{((P3, P2), P4)\} \end{aligned}$$

where $a_i$, $b_i$ are elements of $A_{myP}$, and $P1, P2, \cdots$ are local processes:

# Reversibility of the encoding - Algorithm

with $((a1, P6), \mathbf{myP}) \in prefP$

$\rightsquigarrow myP = prefP(a1, P6)$ by expansion, $\rightsquigarrow myP = a1 \rightarrow P6$ by rewriting

# Reversibility of the encoding - Algorithm

with $((a1, P6), \mathbf{myP}) \in prefP$

$\rightsquigarrow myP = prefP(a1, P6)$ by expansion, $\rightsquigarrow myP = a1 \rightarrow P6$ by rewriting

with $((P5, P0), P6) \in intChoiceP$

$\rightsquigarrow myP = a1 \rightarrow intChoiceP(P5, P0) \rightsquigarrow myP = a1 \rightarrow (P5 \,\square\, prefP(b3, skipP))$

# Reversibility of the encoding - Algorithm

with $((a1, P6), \mathbf{myP}) \in prefP$

$\rightsquigarrow myP = prefP(a1, P6)$ by expansion, $\rightsquigarrow myP = a1 \rightarrow P6$ by rewriting

with $((P5, P0), P6) \in intChoiceP$

$\rightsquigarrow myP = a1 \rightarrow intChoiceP(P5, P0) \rightsquigarrow myP = a1 \rightarrow (P5 \,\Box\, prefP(b3, skipP))$

with $((b3, skipP), P0) \in prefP$

$\rightsquigarrow myP = a1 \rightarrow (P5 \,\Box\, prefP(b3, skipP)) \rightsquigarrow myP = a1 \rightarrow (P5 \,\Box\, (b3 \rightarrow skipP))$

with $((a2, P4), P5) \in prefP$

$\rightsquigarrow myP = a1 \rightarrow (prefP(a2, P4) \,\Box\, (b3 \rightarrow skipP))$

...

with $((b2, skipP), P1) \in prefP$

$\rightsquigarrow myP = a1 \rightarrow$

$. \quad ((a2 \rightarrow ((a2 \rightarrow skipP) \,||\, (a2 \rightarrow prefP(b2, skipP)))) \,\Box\, (b3 \rightarrow skipP))$

$\rightsquigarrow myP = a1 \rightarrow$

$. \quad ((a2 \rightarrow ((a2 \rightarrow skipP) \,||\, (a2 \rightarrow (b2 \rightarrow skipP)))) \,\Box\, (b3 \rightarrow skipP))$



# Encoding of the CSP Semantics rules

$$\frac{myP = \alpha \rightarrow rP}{myP \xrightarrow{\alpha} rP} \ (prefixRule)$$

$$\frac{myP = \alpha \rightarrow rP}{enabledEvent = \alpha \qquad myP' = rP} \ (evolveOnPrefix_0)$$

$$\frac{((\alpha, rP), myP) \in prefP}{\begin{array}{c} trace = trace \frown \alpha \\ prefP = \{(\alpha, rP)\} \lhd prefP \qquad myP = rP \end{array}} \ (evolveOnPrefix)$$

⚠ when prefP $=\{\}$ we use cprefP instead, to deal with recursion.

# CSP Semantics rules: rewriting in Event-B

$$\frac{((\alpha, rP), myP) \in cprefP}{\begin{array}{c}\text{trace} = \text{trace} \frown \alpha \\ cprefP = \{(\alpha, rP)\} \lhd cprefP \qquad myP = rP\end{array}} \quad (evolveOnPrefix)$$

is rewritten as

event evolveOnPrefix
when
$((\alpha, rP), myP) \in cprefP$
then
trace := trace $\frown$ $\alpha$
$cprefP := \{(\alpha, rP)\} \lhd cprefP$
myP := rP
end

# Encoding of CSP Semantics rules: Choice

$$\frac{\begin{array}{c}((P,Q),myP) \in cintChoiceP \\ ((\alpha,rP),P) \in cprefP \qquad ((\beta,rQ),Q) \in cprefP\end{array}}{\begin{array}{c}trace = trace \frown \alpha \\ cprefP = \{(\alpha,rP)\} \lhd cprefP \qquad myP = rP\end{array}} (intChoiceLeft)$$

which is **reversible**!

$$\frac{\begin{array}{c}myP = (P \ \Box \ Q) \\ P = (\alpha \rightarrow rP) \qquad Q = (\beta \rightarrow rQ)\end{array}}{myP \xrightarrow{\alpha} rP} \qquad (intChoiceLeft)$$

# Encoding of CSP Semantics rules: parallel compo.

$$\frac{\begin{array}{c}((P,Q),myP) \in csynParallelP \\ \alpha \in A_P \cap A_Q \\ ((\alpha,rP),P) \in cprefP \qquad ((\alpha,rQ),Q) \in cprefP\end{array}}{\begin{array}{c}trace = trace \frown \alpha \\ csynParallelP = (\{(P,Q)\} \lhd csynParallelP) \\ \cup\ \{((rP,rQ),myP)\}\end{array}}\ (preOnParallelP)$$

which is **reversible**! by rewriting:

$$\frac{\begin{array}{cc}E = A_P \cap A_Q & \alpha \in E \\ P \xrightarrow{\alpha} rP & Q \xrightarrow{\alpha} rQ\end{array}}{P|E|Q \xrightarrow{\alpha} rP|E|rQ}\ (preOnParallelP)$$

# Encoding of CSP Semantics rules

Similar rules are defined for the other operators.

| | |
|---|---|
| Process [] Process | extChoiceP semantic rules |
| Process $\|E\|$ Process | synParallelP semantic rules |
| Process \|\| Process | intParallelP semantic rules |

# Encoding of CSP Semantics rules: recursion

One rule that enables us to handle the recursive case is:

$$\frac{myP \notin \mathrm{ran}(cprefP) \qquad myP \in \mathrm{ran}(prefP)}{cprefP = cprefP \ \cup \ (prefP \ \rhd \{myP\})} \qquad (recursionOnPrefix)$$

# Encoding of CSP Semantics rules: correctness

- The **encoding** of the processes is **reversible**.
- The encoding of the semantics rules is **reversible**.

Applying the same semantic rules to the same processes

# Embedding in a generic Event-B machine

MACHINE PASemantics
SEES Process_Ctx
VARIABLES ...
INVARIANTS
cprefP : $P \times A \leftrightarrow$ P
cintChoiceP : (P × P) → P
cextChoiceP : (P × P) → P
csynParallelP : (P × P) → P
cintParallelP : (P × P) → P
...
EVENTS
initialisation = Subst
(cont'd)
...

.

EVENTS /* sem. rules*/
evolveOnPrefix = ···
preOnParallelP = ···
intChoiceLeft = ···
intchoiceRight = ···
recursionOnPrefix ···
···
END

# Implementation: a generic Event-B framework

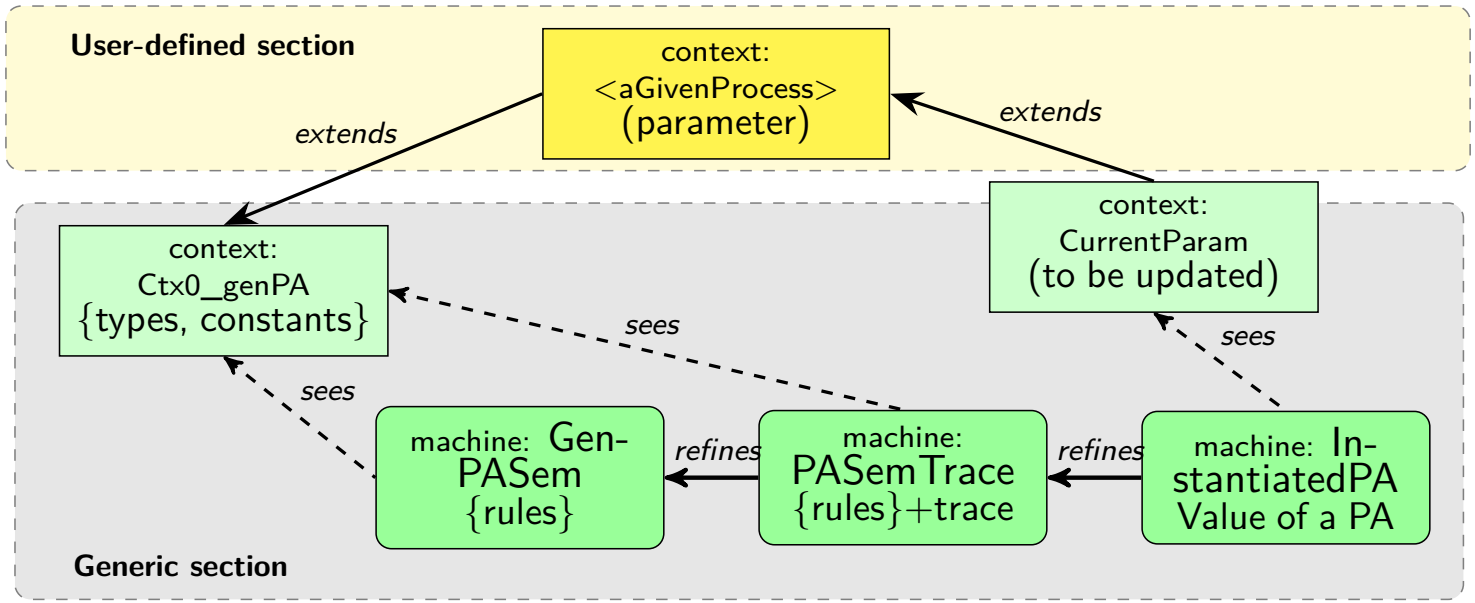



Figure: Architecture of machines

# Implementation: Rodin

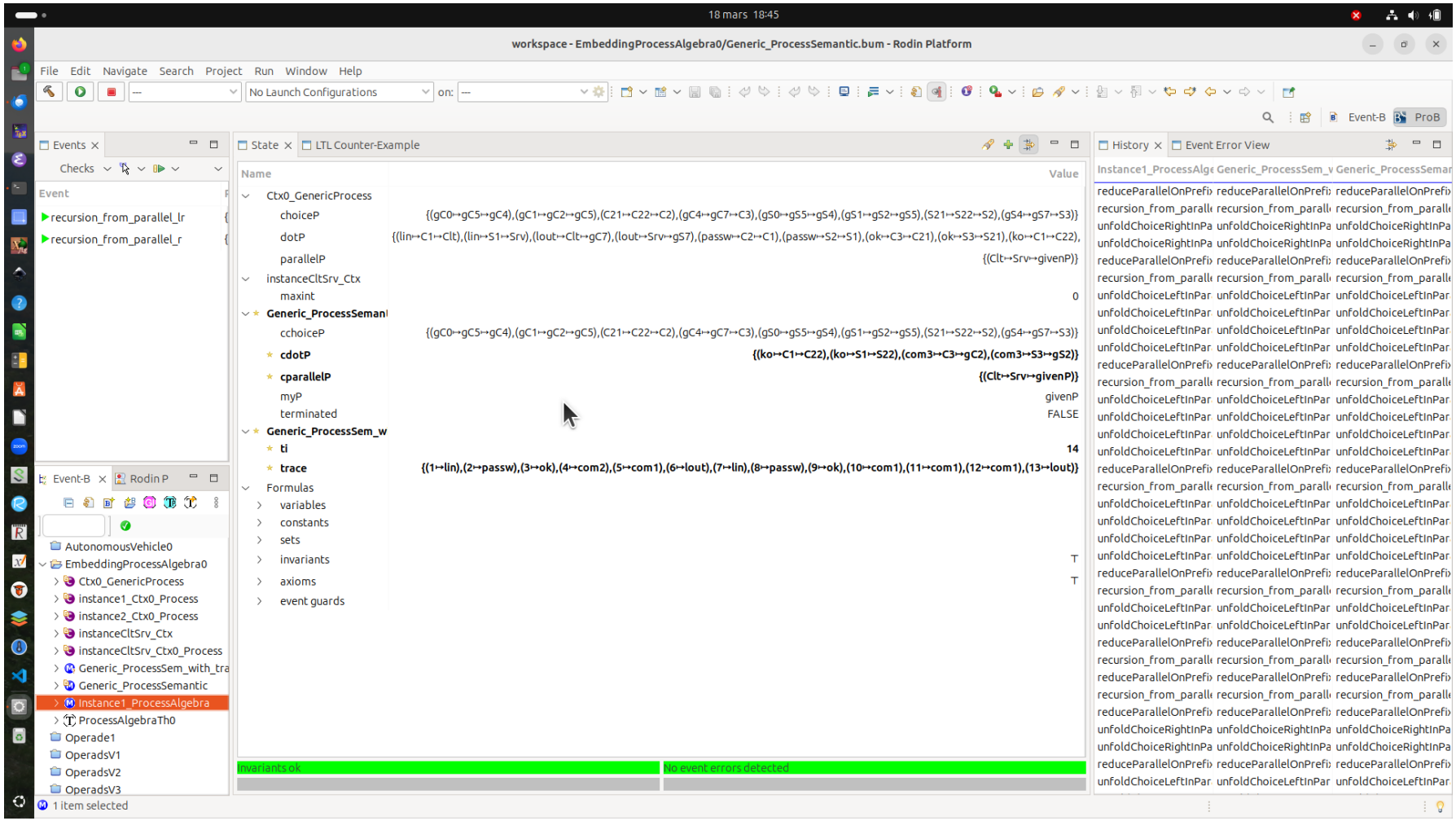



Figure: A behavioural trace of a C/S model

# Experimentations

Various examples (first, manually) :

- Elementary processes
- Composition of processes :

  Client/Server

  Dining philosophers

  ...

Mechanization (Xtext) for more complex case studies

# Conclusion

- ▷ Correct embedding of Process Algebras (CSP) in Event-B
- ▷ Encoding of the syntax of processes and their semantics rules
- ▷ Reversible encodings to ensure correctness
- ▷ Implementation with an Event-B generic framework
- ▷ Experimentations with Rodin

. . . Work still in progress

- ▷ To cover other communications through channels
- ▷ composition of different nature components

Thank you for your attention!